

# Simple Data Structures - 2

## Linked Lists

Y.N. Srikant

Visiting Professor  
Chanakya University  
(Formerly) Professor  
Computer Science and Automation  
Indian Institute of Science  
Bangalore 560 012

# List ADT

A List  $L$  consists of a sequence of elements with the following operations:

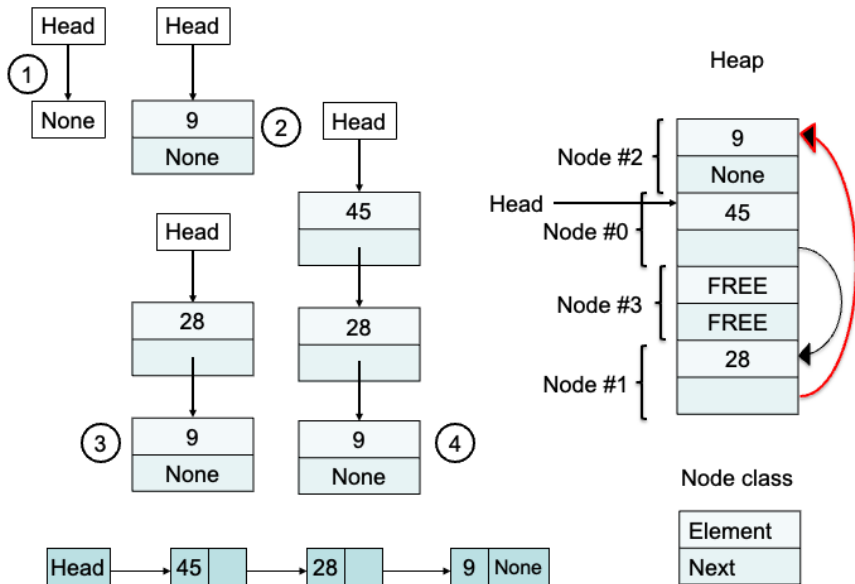
- 1 INSERT( $x, p, L$ ) - Insert  $x$  at position  $p$  in  $L$
- 2 LOCATE( $x, L$ ) - Return position of  $x$  in  $L$
- 3 RETRIEVE( $p, L$ ) - Return element at  $p$  in  $L$
- 4 DELETE( $p, L$ ) - Delete element at position  $p$  in  $L$
- 5 NEXT( $p, L$ ) - Return position following  $p$  in  $L$
- 6 PREVIOUS( $p, L$ ) - Return position preceding  $p$  in  $L$
- 7 MAKENULL( $L$ ) - Make an empty list  $L$
- 8 FIRST( $L$ ) - Return first position in  $L$
- 9 PRINT( $L$ ) - prints list elements in order

A list can be implemented efficiently using linked lists. Array implementation will be inefficient. Some operations will be implemented “inline”.

# Why Linked Lists? Why not only Arrays?

- Consider a long sorted list of integers stored in an array.  
Ex: 100 200 300 400 500.
- To insert 10 into this array, all the numbers have to be shifted by one position. The new array will be: 10 100 200 300 400 500. A similar comment holds for the deletion of 100.
- If these numbers are stored in a linked list, then shifting would have been unnecessary.
- Linked lists allow insertion/deletion of an element at any position without any shifting.

# Linked List Creation



# Creating Linked Lists - Python Version

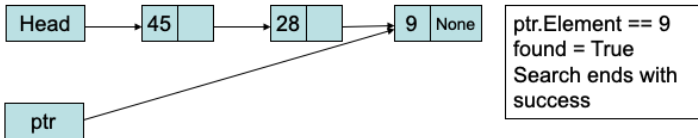
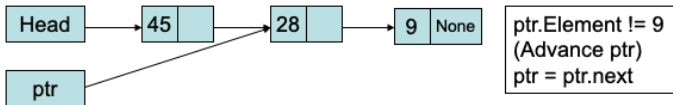
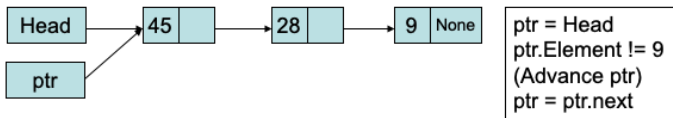
```
class Node:
    def __init__(self,initdata):
        self.Element= initdata
        self.next = None

Head = None
n = int(input("input no.of elements: "))
i = 0
while i<n:
    Element = int(input("input the element: "))
    ptr = Node(Element)
    ptr.next = Head
    Head = ptr
    i = i+1

ptr = Head
while ptr != None:
    print(ptr.Element)
    ptr = ptr.next
```

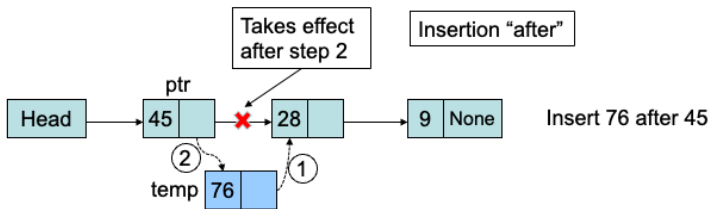
# Linked List Search

Searching for element 9

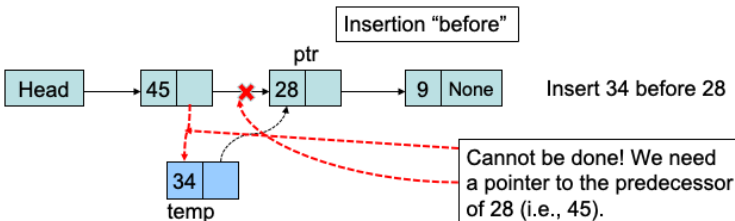


A search for element 20 goes one step further, makes `ptr == None`, and ends the search with a failure.

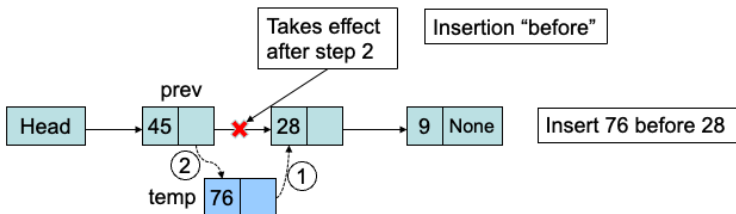
# Linked List Insertion - "After"



temp = Node (76)  
temp.next = ptr.next ①  
ptr.next = temp ②

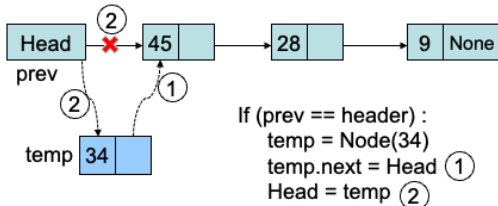


# Linked List Insertion - "Before"



temp = Node (76)  
temp.next = prev.next ①  
prev.next = temp ②

Note that we can work with just the previous node.



If (prev == header) :  
temp = Node(34)  
temp.next = Head ①  
Head = temp ②

# Search and Insertion (“after”) in Linked Lists

```
element = int(input("Insert after what? "))
insert_ele = int(input("New value? "))
ptr = Head
found = False
while ptr != None:
    if (ptr.Element == element):
        found = True
        break
    else:
        ptr = ptr.next

if (found):
    print("Found ", element, ptr.Element)
    temp = Node(insert_ele)
    temp.next = ptr.next
    ptr.next = temp
else:
    print("Did not find ", element)
```

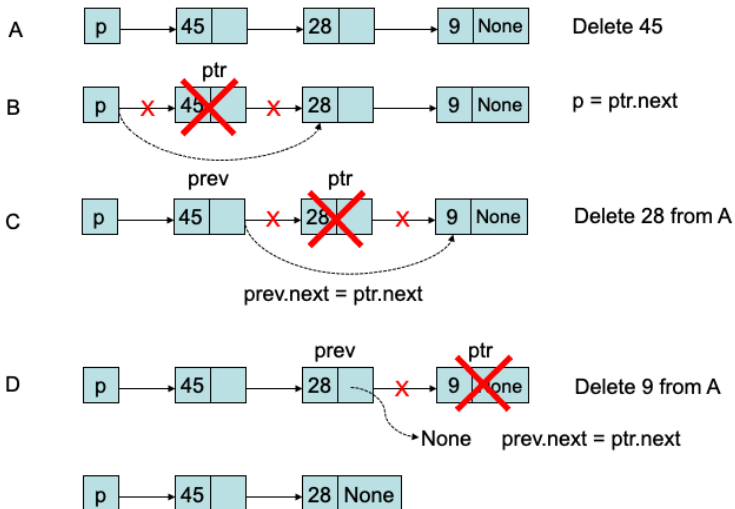
# Search and Insertion (“before”) in Linked Lists

```
element = int(input("Insert before what? "))
insert_ele = int(input("New value? "))
ptr = Head; prev = Head; found = False
while ptr != None:
    if (ptr.Element == element):
        found = True
        break
    else:
        prev = ptr; ptr = ptr.next

if (found):
    print("Found ", element, ptr.Element)
    temp = Node(insert_ele)
    if (ptr==Head):
        temp.next = Head; Head = temp
    else:
        temp.next = prev.next; prev.next = temp
else:
    print("Did not find ", element)
```

# Linked List Deletion - Python Version

## Deletion in Linked Lists – Python Version



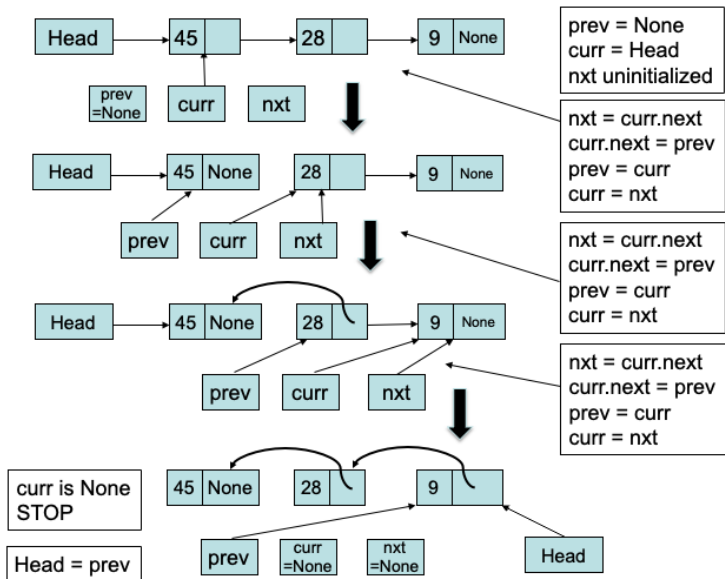
# Deletion in Linked Lists - Python Version

```
#Delete multiple instances also
element = int(input("Delete What? "))
ptr = Head
prev = Head
while ptr != None:
    if (ptr.Element == element):
        if (ptr == Head):
            Head = ptr.next
            ptr = Head
            prev = Head
        else:
            prev.next = ptr.next
            ptr = prev.next
    else:
        prev = ptr
        ptr = ptr.next
```

# Linked List Reversal - Python Version

```
def reverse(head):  
    prev = None  
    curr = head  
  
    while curr is not None:  
        nxt = curr.next  
        curr.next = prev    # Reverse the link  
        prev = curr        # Move prev and  
        curr = nxt         # curr forward  
  
    return prev            # New head (reversed list)
```

# Linked List Reversal - Python Version



# Linked List manipulation with functions in Python (self-reading)

Python passes all function parameters by reference and hence does not need “double pointers” like in C.

```
class Node:
    def __init__(self, initdata):
        self.Element= initdata
        self.next = None
```

```
class LL:
    def __init__(self):
        self.Head = None

    def add_element(self, x):
        ptr = Node(x)
        ptr.next = self.Head
        self.Head = ptr
```

# Linked List manipulation with functions in Python (self-reading)

```
def delete_element(self, x):  
    #Delete multiple instances also  
    ptr = self.Head  
    prev = self.Head  
    while ptr != None:  
        if (ptr.Element == x):  
            if (ptr == self.Head):  
                self.Head = ptr.next  
                ptr = self.Head  
                prev = self.Head  
            else:  
                prev.next = ptr.next  
                ptr = prev.next  
            print("Element ", x, "deleted")  
        else:  
            prev = ptr  
            ptr = ptr.next
```

# Linked List manipulation with functions in Python (self-reading)

```
def create_LL(LHead):  
    n = int(input("input no.of elements: "))  
    i = 0  
    while i<n:  
        Element = int(input("input the element: "))  
        LHead.add_element(Element)  
        i = i+1  
  
LList = LL()  
create_LL(LList)  
ptr = LList.Head  
while ptr != None:  
    print(ptr.Element)  
    ptr = ptr.next
```

# Linked List manipulation with functions in Python (self-reading)

```
n = int(input("input no.of elements to be deleted: "))
i = 0
while i < n:
    Element = int(input("Delete what? "))
    LList.delete_element(Element)
    i = i + 1
ptr = LList.Head
while ptr != None:
    print(ptr.Element)
    ptr = ptr.next
```

# Homework: Not to be submitted

- 1 Write programs in Python to read numbers from the input and insert them into a linked list so that the list remains sorted after each insertion. Note that insertion may happen at the beginning, in the middle or at the end of the linked list.
- 2 Given two sets of integers as two linked lists (arranged in ascending order), write programs in Python to perform set union, set intersection, and set membership.
- 3 **Reading assignment: doubly linked lists and circular lists.**

# A Few Applications of Linked Lists

- 1 Implementation of stacks and queues
- 2 Polynomial representation
- 3 Large integer representation
- 4 Heap representation in dynamic storage allocation
- 5 Browser back/forward history (often implemented with doubly linked lists)
- 6 Operating system process lists
- 7 Graph adjacency lists
- 8 Undo/redo functionality in editors
- 9 Tokenizers in Transformers (in LLMs etc.)

# Large Integer Arithmetic

- consider large integers divided into 4-digit integers and such parts stored in a linked list.
- For example, the integer, 12345678901234567890, will be stored in the linked list as,  
1234 → 5678 → 9012 → 3456 → 7890
- In such a linked list, each component will have the appropriate weightage which is stored in another field.
- In this example, the complete linked list will be,  
1234,16 → 5678,12 → 9012,8 → 3456,4 → 7890,0  
(the two fields are separated by a comma) and the number represented is inferred as:  
 $10^{16} \times 1234 + 10^{12} \times 5678 + 10^8 \times 9012 + 10^4 \times 3456 + 7890.$

# Large Integer Class Example

```
class Large_int_Node:
    def __init__(self,initcomponent,initpower):
        self.component = initcomponent
        self.power = initpower
        self.next = None
```

#Creating the linked list for 12345678.

```
c1 = Large_int_Node(1234,4)
```

```
c2 = Large_int_Node(5678,0)
```

```
c1.next = c2
```

**Homework:** Design algorithms to read, add, subtract, and multiply large integers of size upto 20 digits and implement them in Python using linked lists (NOT Python lists).

# Polynomial Representation

- A polynomial may be stored using linked lists.
- As an example, consider a polynomial in a single variate, such as,  $45x^7 - 2x^4 + 5x - 32$ .
- It may be stored using a linked list, by storing the coefficient and degree of each term in the fields of each node.
- For example, the above polynomial may be stored as,  $45,7 \rightarrow -2,4 \rightarrow 5,1 \rightarrow -32,0$  (the two fields are separated by a comma).
- Obviously, this representation of a polynomial is general and may be extended to multi-variate polynomials also.

# Polynomial Class Example

```
class Poly_Node:
    def __init__(self, initcoeff, initdegree):
        self.coeff = initcoeff
        self.degree = initdegree
        self.next = None
```

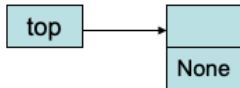
#Creating the linked list for  $45x^7-2x^4-10$ .

```
t1 = Poly_node(45,7)
t2 = Poly_node(-2,4)
t3 = Poly_Node(-10,0)
t1.next = t2
t2.next = t3
```

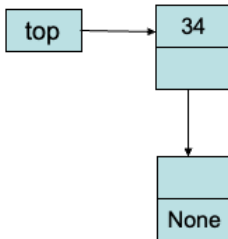
**Homework:** Design algorithms to read, add, subtract, and multiply 2-variate polynomials and implement them in Python using linked lists (NOT Python lists). Assume that the coefficients are all integers.

# Stack with Linked Lists - Python Version

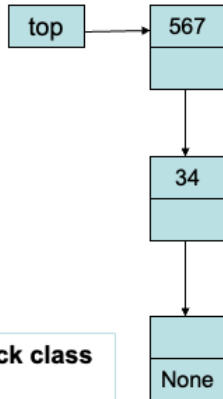
1. Empty stack  
after `s = Stack()`



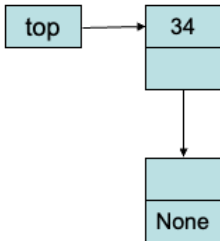
2. After `s.push(34)`



3. After `s.push(567)`



4. After `s.pop()`



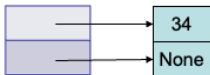
**Homework: Implement a Stack class  
with linked lists**

# Queue with Linked Lists - Python Version

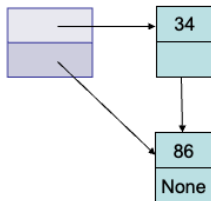
1. Empty Q after  
`q = Queue( )`

front    None  
rear    None  
queue

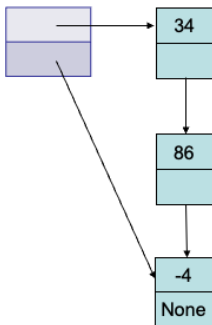
2. After `q.enqueue(34)`



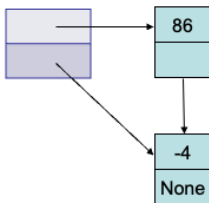
3. After `q.enqueue(86)`



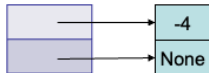
4. After `q.enqueue(-4)`



5. After `q.dequeue( )`



6. After `q.dequeue( )`



**Homework: Implement a Queue class with linked lists**

# When to use Arrays/Linked Lists?

Use arrays for applications with:

- Numerical computing
- Machine learning tensors
- Image processing
- Matrix operations
- Random indexing

Use linked lists for applications with:

- Frequent insertions/deletions
- Unknown size
- Sparse dynamic structures
- Size changes often